भारतीय सूचना प्रौद्योगिकी संस्थान भागलपुर
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHAGALPUR
(An Institute of National Importance under Act of Parliament)

Azadi Ka Amrit Mahotsav

COMPUTER NETWORK ASSIGNMENT

TCP – Sequence Numbers & Acknowledgments

Byte-Oriented Numbering • Wrap-Around Concept

Cumulative ACK • Piggybacking

Name :- Anurag Jaiswal

Roll no-230101034

Group: A Branch :- Cse

1. Introduction to TCP

The Transmission Control Protocol (TCP) is a connection-oriented, reliable, byte-stream transport-layer protocol defined in RFC 793. It operates at Layer 4 (Transport Layer) of both the OSI and TCP/IP models. TCP guarantees that data sent from one application reaches the destination application correctly, completely, and in order — even across a network that may lose, duplicate, or reorder packets.

TCP achieves reliability through a combination of four core mechanisms: (1) Byte-oriented sequence numbering, (2) Acknowledgment numbers, (3) Cumulative ACKs, and (4) Piggybacking. Together, these mechanisms form the backbone of TCP's dependable delivery service.

1.1 TCP Segment Header Overview

Before diving into sequence numbers, it is important to understand where these fields live within a TCP segment. The diagram below shows the TCP header structure:

Source Port (16 bits)	**Destination Port (16 bits)**		
Sequence Number (32 bits)			
Acknowledgment Number (32 bits)			
Data Offset	**Reserved**	**Flags (URG ACK PSH RST SYN FIN)**	**Window Size**
Checksum (16 bits)	**Urgent Pointer (16 bits)**		
Options (variable) + Data			

Key fields relevant to this assignment:

- Sequence Number (32 bits): identifies the position of the first byte of data in this segment
- Acknowledgment Number (32 bits): the next byte the receiver expects to receive
- ACK Flag: must be set to 1 for the Acknowledgment Number field to be valid

2. Byte-Oriented Numbering

2.1 Definition

DEFINITION: Byte-Oriented Numbering
TCP treats data as a continuous stream of bytes. Every single byte in the stream is assigned a unique sequence number. Unlike message-based or segment-based numbering (e.g., UDP), TCP numbers individual bytes — giving it fine-grained control over ordering and retransmission.
The 32-bit Sequence Number (SEQ) field in the TCP header identifies the position (offset) of the FIRST BYTE of data carried in that segment within the entire byte stream.
The initial sequence number (ISN) is randomly chosen during the TCP three-way handshake to enhance security and avoid conflicts with old connections.

2.2 Logic & Working

Suppose Host A wants to send a 3000-byte file to Host B. TCP may split this into multiple segments. Each segment carries a SEQ number corresponding to the byte offset of its first data byte. If the ISN (Initial Sequence Number) is 0:

Segment	Details (ISN = 0)
Segment 1	SEQ = 0, carries bytes 0–999 (1000 bytes)
Segment 2	SEQ = 1000, carries bytes 1000–1999 (1000 bytes)
Segment 3	SEQ = 2000, carries bytes 2000–2999 (1000 bytes)

Example — Byte-Oriented Numbering

Imagine posting 3000 numbered pages of a book one by one. Each envelope is labelled with the page number of its first page. Even if envelopes arrive out of order, the receiver reassembles them correctly by sorting on the label. That label is the sequence number.

```
SENDER (Host A)                              RECEIVER (Host B)
_______________                              _________________

Send Bytes 0-999          ------------------>  Receives Bytes 0-999
SEQ=0, LEN=1000                                ACK=1000 (next expected)

Send Bytes 1000-1999      ------------------>  Receives Bytes 1000-1999
SEQ=1000, LEN=1000                             ACK=2000

Send Bytes 2000-2999      ------------------>  Receives Bytes 2000-2999
SEQ=2000, LEN=1000                             ACK=3000

                          <------------------  Cumulative ACK=3000
```

```
                                                    (Confirms all bytes up to 2999)
```

2.3 Why Random ISN?

Security: A random ISN prevents attackers from guessing sequence numbers (TCP hijacking attacks).
Collision Avoidance: It avoids confusion between old and new connections on the same port pair.

3. Wrap-Around Concept

3.1 Definition

DEFINITION: Wrap-Around
Since the TCP Sequence Number field is 32 bits wide, it can hold values from 0 to 2^32 − 1 (i.e., 0 to 4,294,967,295). Once the sequence number reaches its maximum value, it rolls back (wraps around) to 0 and continues counting. This circular numbering is called the Wrap-Around.
With modern high-speed networks, this wrap-around can happen very quickly. For example, on a 10 Gbps link, the entire 4 GB sequence space can be exhausted in less than 3.4 seconds, making wrap-around a real concern.

3.2 Logic & Working

TCP handles wrap-around transparently — both sender and receiver perform modular arithmetic modulo 2^32. Any comparison of sequence numbers uses modular comparison to determine which number is 'ahead' in the sequence.

```
Sequence Number Space: 0 to 2^32 - 1  (0 to 4,294,967,295)


┌──────────────────────────────────────────────────────────┐
│  0 ──────────────────────────────────── 4,294,967,295   │
│  │                                              │         │
│  └──────────────── WRAP AROUND ◀────────────────┘         │
│                                                           │
│  After SEQ=4,294,967,295, next SEQ = 0 (rolls over)      │
└──────────────────────────────────────────────────────────┘


Example:
SEQ = 4,294,967,000  →  Send 300 bytes
Bytes: 4294967000 to 4294967295  +  Bytes: 0 to 3  (wraps!)
Next expected ACK = 4  (after wrap-around)
```

Example — Wrap-Around

Scenario: Suppose the current SEQ = 4,294,967,100 and the sender needs to send 500 bytes.

- Bytes 4,294,967,100 to 4,294,967,295 → last 196 bytes fit before overflow (196 bytes)
- Remaining 304 bytes → assigned SEQ = 0, 1, 2, ... 303 (after wrap)
- Receiver sends ACK = 304, confirming receipt of 500 bytes across the boundary

3.3 PAWS — Protection Against Wrapped Segments

To handle wrap-around in high-speed networks, RFC 1323 introduced PAWS (Protection Against Wrapped Segments). TCP adds a Timestamp option to each segment. The receiver discards any segment whose timestamp is older than the most recent segment, preventing old, wrapped-around segments from being accepted as fresh data.

4. Cumulative Acknowledgment (ACK)

4.1 Definition

DEFINITION: Cumulative ACK
An Acknowledgment (ACK) in TCP is the receiver's way of telling the sender: 'I have received all bytes up to byte N−1; please send me byte N next.' The ACK number equals the NEXT byte expected by the receiver.
The key property of TCP's ACK is that it is CUMULATIVE — a single ACK confirms receipt of all bytes from the beginning of the stream up to (but not including) the ACK number. This means missing intermediate segments block higher ACKs, and one ACK can confirm many segments at once.

4.2 ACK Number Formula

```
ACK Number = SEQ Number of last received byte + 1
           = Next expected byte from the sender

If SEQ=1000 and LEN=500 was received successfully:
ACK = 1000 + 500 = 1500  (next byte expected is byte #1500)
```

4.3 Logic & Working — Cumulative Nature

Suppose three segments arrive at the receiver: Segment 1 (SEQ=0), Segment 2 (SEQ=1000), Segment 3 (SEQ=2000), each carrying 1000 bytes. If the receiver sends only ONE ACK after receiving all three, that single ACK=3000 implicitly confirms all 3000 bytes. This is cumulative acknowledgment.

```
Sender transmits 3 segments out-of-order receipt scenario:

```

```
Segment 1: SEQ=0,    LEN=1000  → Received ✓
Segment 2: SEQ=1000, LEN=1000  → LOST ✗
Segment 3: SEQ=2000, LEN=1000  → Received (out of order, buffered)

Receiver sends: ACK=1000  (cumulative - only confirms up to 999)
Receiver CANNOT send ACK=3000 because segment 2 is missing!

Sender retransmits Segment 2: SEQ=1000, LEN=1000 → Received ✓
Receiver now sends: ACK=3000  (all bytes 0-2999 confirmed in one shot)

▶ ONE ACK covers ALL previously unacknowledged bytes = Cumulative ACK
```

Example — Lost Segment & Cumulative ACK

Scenario: Segments 1, 2, 3 are sent. Segment 2 is lost in the network.

Event	Explanation
Seg 1 (SEQ=0) arrives	Receiver sends ACK=1000 (cumulative up to byte 999)
Seg 2 (SEQ=1000) is LOST	Receiver gets nothing; waits
Seg 3 (SEQ=2000) arrives	Receiver buffers it; sends ACK=1000 again (duplicate ACK — segment 2 still missing)
Sender retransmits Seg 2	Receiver fills the gap
All data present	Receiver sends ACK=3000 — one ACK for ALL buffered data

4.4 Delayed ACK

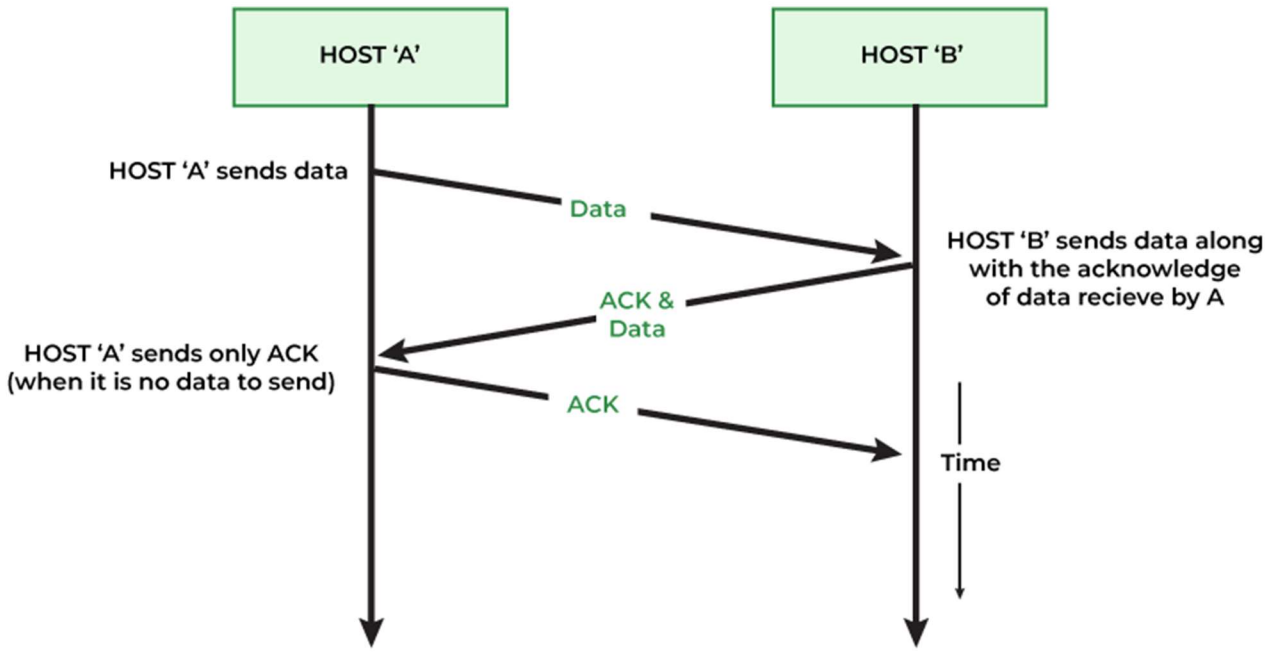
To reduce network overhead, TCP implementations often use Delayed ACK (RFC 1122). Instead of sending an ACK for every segment, the receiver waits up to 200–500 ms hoping to piggyback the ACK on a data segment going in the reverse direction. If no data comes, it sends the ACK alone.

5. Piggybacking

5.1 Definition

DEFINITION: Piggybacking
Piggybacking is the technique of attaching (or 'piggybacking') an acknowledgment for received data onto an outgoing data segment, rather than sending the ACK as a separate, standalone message.
In full-duplex TCP connections, both sides are simultaneously sending data. When Host A sends data to Host B, Host B may have its own data to send back. Instead of sending two separate

messages (one data, one ACK), Host B combines its ACK for A's data with its own outgoing data into a single TCP segment — saving bandwidth and reducing overhead.



5.2 Logic & Working

Piggybacking exploits the full-duplex nature of TCP. The ACK field is always present in every TCP segment. When a receiver has data to send, it simply sets the ACK flag and fills in the ACK number in its outgoing data segment — no extra packet needed.

```
WITHOUT Piggybacking:                    WITH Piggybacking:
_____________________                    __________________
Host A ──[DATA]──────▶ Host B            Host A ──[DATA]──────────────▶ Host B
Host A ◀──[ACK]──────── Host B           Host B ──[DATA + ACK piggybacked]▶ Host A
Host A ──[DATA]──────▶ Host B
Host A ◀──[ACK]──────── Host B

4 messages exchanged                     2 messages exchanged
(2 separate ACKs sent)                   (ACK rides along with DATA)

▶ Piggybacking reduces overhead by combining ACK with outgoing data!
```

Example — Piggybacking in Action

Scenario: Host A is a client requesting a webpage from Host B (server). Both sides are talking simultaneously.

Message	What it contains
A → B	HTTP GET request (DATA: SEQ=1000, LEN=100)
B → A	HTTP 200 OK response (DATA: SEQ=5000) + ACK=1100 piggybacked
A → B	Next HTTP request (DATA) + ACK=5500 piggybacked

5.3 Benefits of Piggybacking

- Reduces number of packets on the network → less congestion
- Saves bandwidth — no separate ACK packets
- Improves throughput by combining control and data
- Works seamlessly with TCP's Delayed ACK mechanism

5.4 When Piggybacking Cannot Occur

- Unidirectional data transfer (only one side sends data)
- When the receiver has no data to send in the reverse direction
- Timeout forces standalone ACK before outgoing data is ready

6. Concept Summary

Concept	Key Idea	Key Formula / Example
Byte-Oriented Numbering	Each byte has a unique sequence number	`SEQ=1000 → first byte is byte #1000`
Wrap-Around	32-bit SEQ rolls from 4,294,967,295 back to 0	`After max, SEQ = 0 (modular arithmetic)`
Cumulative ACK	One ACK confirms ALL bytes up to ACK−1	`ACK=3000 confirms bytes 0-2999`
Piggybacking	ACK rides on outgoing data segment	`DATA + ACK in one segment → fewer packets`

7. Previous Year Questions (GATE / UGC-NET)

The following questions are representative of the types of MCQs asked in GATE CS and UGC-NET examinations on the topic of TCP Sequence Numbers and Acknowledgments. Each question is followed by all options and a detailed explanation of why the correct answer is right and why the others are wrong.

MCQ 1 — GATE CS 2014 (Similar Pattern)

Q. Consider a TCP connection between Host A and Host B. The Initial Sequence Number (ISN) at Host A is 200. Host A sends a segment containing 100 bytes of data. Host B acknowledges this segment. What will be the Acknowledgment Number sent by Host B?

This question tests understanding of byte-oriented numbering and ACK calculation.

Opt	Option Text	Remark
A	200	Incorrect — ACK = ISN is only for a SYN (0-byte) segment. A data segment with 100 bytes advances the sequence.
B	**300**	✓ CORRECT
C	301	Incorrect — The ACK = SEQ + LEN = 200 + 100 = 300, not 301. We add 1 only for control segments (SYN/FIN).
D	100	Incorrect — 100 is only the number of bytes, not the ACK number.

Detailed Explanation — MCQ 1

CORRECT ANSWER: (B) 300

The formula for ACK is: ACK = SEQ + Number of data bytes = 200 + 100 = 300.

Host A sends a segment with SEQ=200 and 100 bytes of data (bytes 200 to 299). The LAST byte received is byte #299. The NEXT byte expected is byte #300. Therefore, Host B sends ACK=300.

Common trap: Students often add 1 (ACK = 300+1 = 301) thinking of it like a SYN/FIN segment. But SYN and FIN each consume 1 sequence number even though they carry no data. A regular data segment does NOT get the +1. ACK = SEQ + LEN (for data); ACK = SEQ + 1 (for SYN or FIN only).

MCQ 2 — UGC-NET CS June 2019 (Pattern)

Q. In TCP, the acknowledgment number in a segment sent from receiver to sender indicates:

This question tests the conceptual definition of the TCP Acknowledgment Number field.

Opt	Option Text	Remark
A	The sequence number of the last byte successfully received	Incorrect — If last byte received has SEQ=N, then ACK = N+1, not N. This is a common off-by-one confusion.
B	**The sequence number of the next byte expected by the receiver**	✓ CORRECT
C	The total number of bytes received so far	Incorrect — ACK is not a byte count from zero; it is an absolute byte offset in the stream.
D	The sequence number of the last segment received	Incorrect — Segments can carry multiple bytes; TCP ACKs byte offsets, not segment numbers.

Detailed Explanation — MCQ 2
CORRECT ANSWER: (B) The sequence number of the next byte expected by the receiver
The TCP RFC 793 specification defines: 'The acknowledgment number contains the value of the next sequence number the sender of the segment is expecting to receive.'
Explanation with example: If the receiver has successfully received bytes 0 through 499 (500 bytes), it sends ACK=500. This tells the sender: 'I have everything up to byte 499; please send me byte 500 next.'
Option A is incorrect because if the last received byte is #499, the ACK is 500 (not 499). This off-by-one error is a very common mistake in exams.
Option C is incorrect because the ACK number is an absolute byte offset from the ISN, not a count. If ISN=1000 and 500 bytes were received, ACK=1500 (not ACK=500).
Option D is incorrect because TCP is byte-stream oriented, not segment-oriented. Segments are just a transmission convenience; the sequence space is over bytes.

MCQ 3 — GATE CS 2019 (Pattern) — Wrap-Around
Q. The sequence number field in a TCP segment is 32 bits wide. If the initial sequence number is 0 and data is transmitted at 10 Gbps, approximately how long (in seconds) does it take for the sequence numbers to wrap around?
1 Gbps = 10^9 bits/second. The answer should be rounded to the nearest integer.

Opt	Option Text	Remark
A	About 0.34 seconds	Incorrect — This corresponds to 4 GB / 100 Gbps, not 10 Gbps.
B	**About 3.4 seconds**	✓ CORRECT
C	About 34 seconds	Incorrect — This corresponds to a much slower link speed (~1 Gbps with wrong unit conversion).

D	About 3400 seconds	Incorrect — Orders of magnitude off; would correspond to ~10 Mbps.

Detailed Explanation — MCQ 3
CORRECT ANSWER: (B) About 3.4 seconds
Step-by-step calculation:
Sequence number space = 2^32 bytes = 4,294,967,296 bytes ≈ 4.295 GB
Transmission rate = 10 Gbps = 10 × 10^9 bits/s = 10^9 bytes/s (since 1 byte = 8 bits, 10Gbps = 1.25 × 10^9 bytes/s)
Time = Total bytes / Rate = 4.295 × 10^9 bytes ÷ 1.25 × 10^9 bytes/s ≈ 3.44 seconds
This illustrates why PAWS (Protection Against Wrapped Segments, RFC 1323) and TCP timestamps are necessary on high-speed networks. At 10 Gbps, sequence numbers wrap in just ~3.4 seconds — well within the Maximum Segment Lifetime (MSL = 120 seconds), creating a risk of old packets being mistaken for new ones.

MCQ 4 — UGC-NET CS December 2018 (Pattern) — Piggybacking

Q. Which of the following statements about piggybacking in TCP is CORRECT?

Opt	Option Text	Remark
A	Piggybacking means sending multiple data segments without waiting for ACK	Incorrect — This describes pipelining or Go-Back-N, not piggybacking.
B	**Piggybacking attaches an acknowledgment to an outgoing data segment to reduce overhead**	✓ CORRECT
C	Piggybacking is used only in UDP, not TCP	Incorrect — Piggybacking is specifically a TCP (and data link layer) concept. UDP has no ACK mechanism at all.
D	Piggybacking increases the total number of packets transmitted	Incorrect — Piggybacking REDUCES the number of packets by combining ACK with data, not increasing them.

Detailed Explanation — MCQ 4
CORRECT ANSWER: (B) Piggybacking attaches an acknowledgment to an outgoing data segment to reduce overhead
Piggybacking is defined as the practice of including an acknowledgment for received data within an outgoing data segment, rather than sending a separate ACK packet.
Why it works: TCP is full-duplex. Both A→B and B→A data can flow simultaneously. When B has data to send to A, it can include the ACK for A's data in the same segment — saving one complete packet.

Option A describes a sliding window / pipelining mechanism (Go-Back-N, Selective Repeat), not piggybacking.

Option C is wrong because UDP has no connection state and no ACK mechanism. Piggybacking is meaningless in UDP.

Option D is the exact opposite of the truth. The entire point of piggybacking is to REDUCE the number of packets — combining what would be two packets (DATA + standalone ACK) into one.

8. Quick Revision — Key Points for GATE/UGC-NET

#	Important Fact to Remember
1	TCP sequence numbers are byte-oriented, NOT segment-oriented
2	ACK = SEQ + LEN (for data segments); ACK = SEQ + 1 (for SYN/FIN)
3	Sequence number field is 32 bits → range: 0 to 4,294,967,295
4	Wrap-around is handled by modular arithmetic (mod 2^32)
5	PAWS (RFC 1323) uses TCP timestamps to prevent wrap-around issues
6	Cumulative ACK: one ACK implicitly confirms ALL preceding bytes
7	Duplicate ACKs (same ACK number repeated) indicate a lost segment
8	Piggybacking = ACK carried inside an outgoing data segment (reduces packets)
9	Delayed ACK: receiver waits up to 200ms to piggyback ACK onto data
10	TCP three-way handshake uses SYN segments; SYN consumes 1 seq number

— End of Assignment —